The softmax function and its gradient

Siavash Ahmadi

1 The softmax function

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix} \in \mathbb{R}^n$$

$$\text{softmax}(\mathbf{x}) = \begin{bmatrix} \frac{e^{x_1}}{\sum_k e^{x_k}} & \frac{e^{x_2}}{\sum_k e^{x_k}} & \cdots & \frac{e^{x_n}}{\sum_k e^{x_k}} \end{bmatrix}$$

$$\text{softmax}(\mathbf{x})_j = \frac{e^{x_j}}{\sum_k e^{x_k}} = \frac{u(x_j)}{v(\mathbf{x})}$$

where $u(x) = e^x$ and $v(\mathbf{x}) = \sum_k e^{x_k}$.

2 The softmax Jacobian

Recall the quotient derivative rule:

$$\frac{d}{dx}\left(\frac{u(x)}{v(x)}\right) = \frac{u'(x)v(x) - u(x)v'(x)}{[v(x)]^2} \tag{1}$$

We have

$$\frac{\partial}{\partial x_i} u(x_j) = \begin{cases} u(x_i) & , i = j \\ 0 & , i \neq j \end{cases} = \delta_{ij} u(x_i) \tag{2}$$

$$\frac{\partial}{\partial x_i} v(\mathbf{x}) = u(x_i) \tag{3}$$

where $\delta_{ij} = 1$ when $i = j$ and 0 otherwise. Now,

$$\begin{aligned}
\frac{\partial}{\partial x_i}\text{softmax}(\mathbf{x})_j &= \frac{[\frac{\partial}{\partial x_i}u(x_j)]v(\mathbf{x}) - u(x_j)[\frac{\partial}{\partial x_i}v(\mathbf{x})]}{[v(\mathbf{x})]^2} && \text{(by (1))} \\
&= \frac{\delta_{ij}u(x_i)v(\mathbf{x}) - u(x_j)u(x_i)}{v(\mathbf{x})v(\mathbf{x})} && \text{(by (2) and (3))} \\
&= \frac{u(x_i)}{v(\mathbf{x})} \cdot \frac{\delta_{ij}v(\mathbf{x}) - u(x_j)}{v(\mathbf{x})} \\
&= \frac{u(x_i)}{v(\mathbf{x})} \cdot \left(\delta_{ij}\frac{v(\mathbf{x})}{v(\mathbf{x})} - \frac{u(x_j)}{v(\mathbf{x})}\right) \\
&= \frac{u(x_i)}{v(\mathbf{x})} \cdot \left(\delta_{ij} - \frac{u(x_j)}{v(\mathbf{x})}\right) \\
&= \boxed{\text{softmax}(\mathbf{x})_i(\delta_{ij} - \text{softmax}(\mathbf{x})_j)} = \text{softmax}(\mathbf{x})_j(\delta_{ij} - \text{softmax}(\mathbf{x})_i)
\end{aligned}$$

If we set $\mathbf{p} = \text{softmax}(\mathbf{x})$, a vectorized implementation of this could look like this:

```python
def softmax_grad(x):
    p = softmax(x, axis=-1)
    grad = np.diag(p) - p.T @ p
    return grad
```

Here's a horrendously detailed explanation of why this works:

$$\mathbf{p} = \begin{bmatrix} p_1 & p_2 & \cdots & p_n \end{bmatrix}$$

$$\text{diag}(\mathbf{p}) = \begin{bmatrix} p_1 & 0 & \cdots & 0 \\ 0 & p_2 & \cdots & 0 \\ \vdots & & & \vdots \\ 0 & 0 & \cdots & p_n \end{bmatrix}$$

$$\mathbf{p}^\top \mathbf{p} = \begin{bmatrix} p_1 \\ p_2 \\ \vdots \\ p_n \end{bmatrix} \begin{bmatrix} p_1 & p_2 & \cdots & p_n \end{bmatrix} = \begin{bmatrix} p_1 p_1 & p_1 p_2 & \cdots & p_1 p_n \\ p_2 p_1 & p_2 p_2 & \cdots & p_2 p_n \\ \vdots & & & \vdots \\ p_n p_1 & p_n p_2 & \cdots & p_n p_n \end{bmatrix}$$

$$\text{diag}(\mathbf{p}) - \mathbf{p}^\top \mathbf{p} = \begin{bmatrix} p_1 & 0 & \cdots & 0 \\ 0 & p_2 & \cdots & 0 \\ \vdots & & & \vdots \\ 0 & 0 & \cdots & p_n \end{bmatrix} - \begin{bmatrix} p_1 p_1 & p_1 p_2 & \cdots & p_1 p_n \\ p_2 p_1 & p_2 p_2 & \cdots & p_2 p_n \\ \vdots & & & \vdots \\ p_n p_1 & p_n p_2 & \cdots & p_n p_n \end{bmatrix}$$

$$= \begin{bmatrix} p_1 - p_1 p_1 & -p_1 p_2 & \cdots & -p_1 p_n \\ -p_2 p_1 & p_2 - p_2 p_2 & \cdots & -p_2 p_n \\ \vdots & & & \vdots \\ -p_n p_1 & -p_n p_2 & \cdots & p_n - p_n p_n \end{bmatrix}$$

$$= \begin{bmatrix} p_1(1 - p_1) & p_1(0 - p_2) & \cdots & p_1(0 - p_n) \\ p_2(0 - p_1) & p_2(1 - p_2) & \cdots & p_2(0 - p_n) \\ \vdots & & & \vdots \\ p_n(0 - p_1) & p_n(0 - p_2) & \cdots & p_n(1 - p_n) \end{bmatrix}$$

$$= \begin{bmatrix} p_1(\delta_{ij} - p_1) & p_1(\delta_{ij} - p_2) & \cdots & p_1(\delta_{ij} - p_n) \\ p_2(\delta_{ij} - p_1) & p_2(\delta_{ij} - p_2) & \cdots & p_2(\delta_{ij} - p_n) \\ \vdots & & & \vdots \\ p_n(\delta_{ij} - p_1) & p_n(\delta_{ij} - p_2) & \cdots & p_n(\delta_{ij} - p_n) \end{bmatrix}$$

$$= \mathbf{p}_i(\delta_{ij} - \mathbf{p}_j) = \nabla_{x_i} \text{softmax}(\mathbf{x})_j$$

3 The gradient when softmax is an intermediate computation

When we are computing the gradient of an upstream loss with respect to the softmax inputs, something interesting happens. In this case, we don't need to compute the Jacobian first, as seen below:

$$
\begin{aligned}
G &= \nabla_{\mathbf{p}} \mathcal{L} \in \mathbb{R}^{1 \times n} && \text{\# upstream gradient} \\
\frac{\partial \mathcal{L}}{\partial \mathbf{x}} &= \frac{\partial \mathcal{L}}{\partial \mathbf{p}} \cdot \frac{\partial \mathbf{p}}{\partial \mathbf{x}} \in \mathbb{R}^{1 \times n} \\
&= G(\operatorname{diag}(\mathbf{p}) - \mathbf{p}^\top \mathbf{p}) \\
&= \underbrace{G \cdot \operatorname{diag}(\mathbf{p})}_{\text{matrix multiply}} - \underbrace{G \cdot \underbrace{\mathbf{p}^\top \mathbf{p}}_{\text{matrix}}}_{\text{matrix multiply}} \\
&= \underbrace{G \odot \mathbf{p}}_{\text{scalar products}} - \underbrace{\underbrace{(G \cdot \mathbf{p}^\top)}_{\text{dot product}} \mathbf{p}}_{\text{scalar by vector}} \\
&\Rightarrow \boxed{\nabla_{\mathbf{x}} \mathcal{L} = (G - G \cdot \mathbf{p}) \odot \mathbf{p}} \in \mathbb{R}^{1 \times n}
\end{aligned}
$$

So now the code becomes

```
def softmax_grad(x, g):
    p = softmax(x, axis=-1)

    g_dot_p = np.vecdot(g, p)[..., np.newaxis]
    grad    = (g - g_dot_p) * p

    return grad
```